

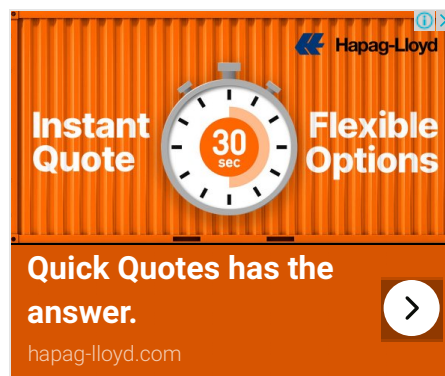


Debug Node.js Application

In this section, you will learn ways to debug Node.js application.

You can debug Node.js application using various tools including following:

1. Core Node.js debugger
2. Node Inspector
3. Built-in debugger in IDEs



Core Node.js Debugger

Node.js provides built-in non-graphic debugging tool that can be used on all platforms. It provides different commands for debugging Node.js application.

Consider the following simple Node.js application contained in app.js file.

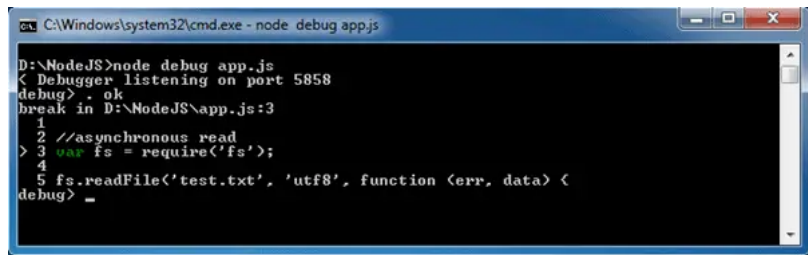
app.js	Copy
<pre>var fs = require('fs'); fs.readFile('test.txt', 'utf8', function (err, data) { debugger; if (err) throw err; console.log(data); });</pre>	

Write debugger in your JavaScript code where you want debugger to stop. For example, we want to check the "data" parameter in the above example. So, write debugger; inside callback function as above.

Now, to debug the above application, run the following command.

```
node debug app.js
```

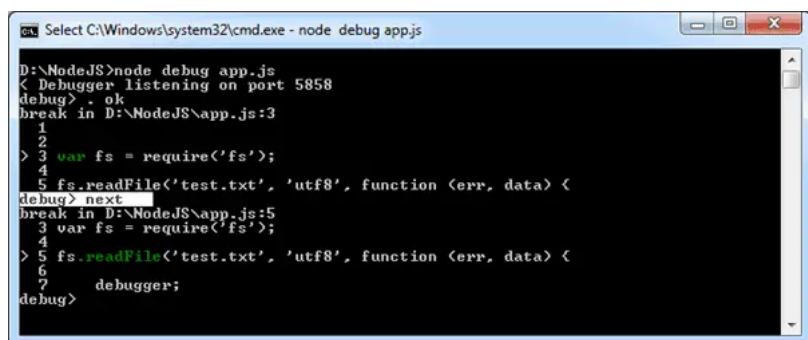
The above command starts the debugger and stops at the first line as shown below.



Starting Node.js Debugging

As you can see in the above figure, > symbol indicates the current debugging statement.

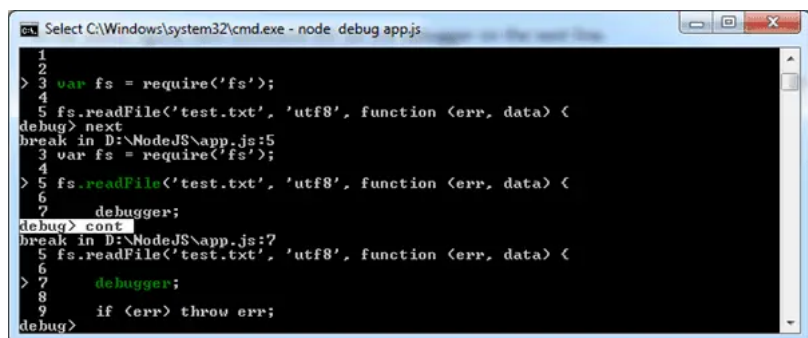
Use **next** to move on the next statement.



Debug Node.js Application

In the above figure, next command will set the debugger on the next line. The > is now pointing to next statement.

Use **cont** to stop the execution at next "debugger", if any.



Debug Node.js Application

In the above figure, you can see that cont command stops at the "debugger".

Use `watch('expression')` command to add the variable or expression whose value you want to check. For example, to check the value of data variable in the above example, add data into watch expression as shown below.

```

Select C:\Windows\system32\cmd.exe - node debug app.js
break in D:\NodeJS\app.js:5
3 var fs = require('fs');
4
> 5 fs.readFile('test.txt', 'utf8', function (err, data) {
6
7   debugger;
debug> cont
break in D:\NodeJS\app.js:7
5 fs.readFile('test.txt', 'utf8', function (err, data) {
6
7   debugger;
8
9   if (err) throw err;
debug> watch("data")
debug>
```

Debug Node.js Application

Now, write watchers command to check the value of all the variables added into watch().

```

Select C:\Windows\system32\cmd.exe - node debug app.js
break in D:\NodeJS\app.js:5
3 var fs = require('fs');
4
> 5 fs.readFile('test.txt', 'utf8', function (err, data) {
6
7   debugger;
debug> cont
break in D:\NodeJS\app.js:7
5 fs.readFile('test.txt', 'utf8', function (err, data) {
6
7   debugger;
8
9   if (err) throw err;
debug> watch("data")
debug> watchers
0: data = "Hello Node.js World!"
debug>
```

Debug Node.js Application

The following table lists important debugging commands:

next	Stop at the next statement.
cont	Continue execute and stop at the debugger statement if any.
step	Step in function.
out	Step out of function.
watch	Add the expression or variable into watch.
watcher	See the value of all expressions and variables added into watch.
Pause	Pause running code.

Thus, you can use built-in Node.js debugger to debug your Node.js application. Visit [Node.js official documentation](#) to know all the [Node.js debugging commands](#) or write "help" in debug mode in the Node.js console (REPL).

Learn how to use Node Inspector to debug Node.js application in the next section.

Want to check how much you know Node.js?

Start Node.js Test